

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO
COORDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

João Pessoa - PB
10/2026

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do Curso Superior de Tecnologia em Sistemas para Internet

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

Relatório de Estágio apresentado ao Coordenação do Curso Superior de Tecnologia em Sistemas para Internet da Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, como requisito parcial para a obtenção do título de Tecnólogo.

Orientador:	Juliana Dantas Ribeiro Viana de Medeiros
Supervisor:	Juliana Dantas Ribeiro Viana de Medeiros
Coordenador do Curso:	Cândido José Ramos do Egypto
Empresa:	Agência de Inovação (NEO-IFPB)
Período:	25/02/2026 à 19/06/2026

João Pessoa - PB

APROVAÇÃO

Esta folha deverá ser substituída pela cópia digitalizada da folha de aprovação fornecida.

Juliana Dantas Ribeiro Viana de Medeiros
Supervisor da Empresa

Cândido José Ramos do Egypto
Coordenador do CST de Sistemas para Internet

Prof. Dr. Juliana Dantas Ribeiro Viana de Medeiros
Professor Orientador

Luis Kilmer da Silva Bernardo
Estagiário

Altere este texto inserindo a dedicatória do seu trabalho.

AGRADECIMENTOS

Edite e coloque aqui os agradecimentos às pessoas e/ou instituições que contribuíram para a realização do trabalho.

É obrigatório o agradecimento às instituições de fomento à pesquisa que financiaram total ou parcialmente o trabalho, inclusive no que diz respeito à concessão de bolsas.

Eu denomino meu campo de Gestão do Conhecimento, mas você não pode gerenciar conhecimento. Ninguém pode. O que pode fazer - o que a empresa pode fazer - é gerenciar o ambiente que otimize o conhecimento. (PRUSAK, Laurence, 1997).

RESUMO

Silva Bernardo, Luis Kilmer. Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas. 10/2026. 40 f. Relatório de Estágio – Coordenação do Curso Superior de Tecnologia em Sistemas para Internet, Instituto Federal de Educação, Ciência e Tecnologia da Paraíba. João Pessoa - PB, 10/2026.

Este relatório apresenta as atividades desenvolvidas no Laboratório Assert do Instituto Federal da Paraíba (IFPB), Campus João Pessoa, entre fevereiro e junho de 2026, por meio do PIAGI. O estágio teve como objetivo desenvolver um módulo para o sistema Banco de Especialistas, visando formalizar e rastrear solicitações de alterações cadastrais e de classificação dos usuários. Para isso, implementou-se o módulo de Solicitações, contemplando a modelagem das entidades Solicitação e Snapshot, as regras de negócio para criação, edição, aprovação e rejeição de solicitações e uma linha do tempo (timeline) para registrar os eventos e as decisões do processo. O desenvolvimento também abrangeu a implementação de operações de persistência e endpoints, a validação e serialização de dados, a integração com a arquitetura existente e a construção de funcionalidades de interface para solicitantes e avaliadores. A solução utilizou Python, Flask, PostgreSQL, SQLAlchemy e Marshmallow no back-end, Nuxt no front-end e Docker e Docker Compose para configuração do ambiente. Também foram desenvolvidos testes automatizados com Pytest para verificar as funcionalidades implementadas. Como resultado, o módulo integrou o fluxo de solicitações à plataforma, reduzindo a dependência de comunicações por e-mail e intervenções manuais no banco de dados, além de proporcionar maior formalização, rastreabilidade e histórico das alterações realizadas.

Palavras-chave: Desenvolvimento full-stack. Banco de Especialistas. Solicitações. Rastreabilidade. Testes automatizados.

LISTA DE FIGURAS

Figura 1 – Fluxo de alteração de categoria de especialistas antes do desenvolvimento do módulo. 12

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais do módulo 15
Quadro 2 – Requisitos não funcionais do módulo 16
Quadro 3 – Endpoints do módulo de solicitações 17
Quadro 4 – Modelo de dados do módulo de solicitações 19
Quadro 5 – Modelo da entidade SpecialistSnapshot 23
Quadro 6 – Distribuição dos testes por arquivo 28

LISTA DE TABELAS

Tabela 1 – Volume de chamados por tipo de inscrição 13
Tabela 2 – Distribuição dos chamados por categoria de problema 13

LISTA DE ALGORITMOS

Algoritmo 1 – Padrão de implementação observado em controladores existentes. . . . 10
Algoritmo 2 – Estrutura do novo módulo, com separação de responsabilidades. 11
Algoritmo 3 – Estrutura da rota de criação de solicitação. 18
Algoritmo 4 – Estrutura da entidade `Solicitation` e enumerações de domínio (pseudocódigo). 20
Algoritmo 5 – Fluxo de reversão (pseudocódigo). 25
Algoritmo 6 – Limpeza de tabelas transacionais entre testes (pseudocódigo). 29

SUMÁRIO

LISTA DE FIGURAS viii

LISTA DE QUADROS ix

LISTA DE TABELAS x

LISTA DE ABREVIATURAS E SIGLAS xi

LISTA DE SÍMBOLOS xi

LISTA DE ALGORITMOS xi

1 – INTRODUÇÃO 1

1.1 Objetivos 1

1.1.1 Objetivo geral 1

1.1.2 Objetivos específicos 2

1.2 Escopo e limitações 2

1.3 Procedimentos metodológicos 2

1.4 Contexto institucional 3

1.5 Organização do relatório 3

2 – EMBASAMENTO TÉORICO 4

2.1 Arquitetura Web Cliente-Servidor 4

2.2 Flask 4

2.3 Persistência de dados com PostgreSQL e SQLAlchemy 4

2.3.1 PostgreSQL 5

2.3.2 SQLAlchemy 5

2.4 Validação de Dados com Marshmallow e Flasgger 5

2.5 Auditoria, Rastreabilidade e Versionamento de Dados 5

2.5.1 Auditoria e rastreabilidade 5

2.5.2 Fluxo de aprovação com máquina de estados 6

2.5.3 Snapshot e reversão 6

2.5.4 Timeline e registro de eventos 7

2.6 Verificação das Transições com Pytest 7

2.7 Ambiente Reprodutível de Testes com Docker 7

2.8 Controle de Versão e Gestão de Configuracao 7

2.9 Síntese do Capítulo 8

3 – ATIVIDADES REALIZADAS 9
3.1 Estudo da Arquitetura do Sistema Existente 9
3.1.1 Visão Geral do Sistema 9
3.1.2 Características do Código Existente e Decisão por Additive Change . 9
3.1.3 Análise das Regras de Negócio 12
3.1.4 Entendimento do Fluxo Antigo e Diagnóstico de Demanda 12
3.1.5 Levantamento de Requisitos 14
3.1.5.1 Requisitos Funcionais 14
3.1.5.2 Requisitos Não Funcionais 16
3.2 Desenvolvimento do Módulo de Solicitações 17
3.2.1 Endpoints 17
3.2.2 Integração com o Banco de Dados 18
3.2.3 Autorização e Controle de Acesso 22
3.2.4 Aprovação e Rejeição 22
3.2.5 Mudança de categoria 22
3.2.6 Atualização de pontuação 22
3.3 Implementação do Sistema de Snapshot e Auditoria 23
3.3.1 Entidade Snapshot e Modelo de Persistência 23
3.3.2 Reversão de Alterações 24
3.3.3 Linha do Tempo de Eventos 26
3.3.4 Rastreabilidade e Limitações 26
3.4 Testes e Validação 27
3.4.1 Construção da Infraestrutura de Testes 27
3.4.2 Isolamento entre Testes 29
3.4.3 Fixtures de Autenticação 30
3.4.4 Testes de integração 30
3.4.5 Processo de Integração Contínua 31

4 – CONSIDERAÇÕES FINAIS 32

Referências 33

Apêndices 35

APÊNDICE A – Nome do apêndice 36

APÊNDICE B – Nome do outro apêndice 37

Anexos 38

ANEXO A–Nome do anexo 39

ANEXO B–Nome do outro anexo 40

1 INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio supervisionado realizado entre fevereiro e junho de 2026 no Laboratório Assert, vinculado ao Polo de Inovação do Instituto Federal da Paraíba (IFPB). As atividades concentraram-se na evolução do sistema Banco de Especialistas, plataforma destinada ao gerenciamento de especialistas e ao apoio à composição de equipes para projetos de Pesquisa, Desenvolvimento e Inovação (PD&I).

O Banco de Especialistas permite o cadastro, a classificação e a avaliação de candidatos com base em critérios como formação acadêmica, experiência profissional e participação em projetos. Nesse contexto, a plataforma reúne informações utilizadas na gestão dos especialistas e nos processos de seleção e composição de equipes vinculadas às iniciativas do Polo de Inovação.

Durante o estágio, identificou-se uma demanda relacionada à alteração de categoria dos especialistas. O procedimento então utilizado exigia que o interessado solicitasse a mudança por e-mail e que um administrador realizasse manualmente uma consulta SQL diretamente no banco de dados. A ausência de um fluxo formal de solicitações dificultava o acompanhamento das alterações e a recuperação do estado anterior dos registros, além de aumentar o risco de inconsistências decorrentes de intervenções diretas na base de dados.

Diante desse cenário, o trabalho concentrou-se no desenvolvimento de um módulo de solicitações para formalizar e gerenciar operações de alteração cadastral e de classificação dos especialistas. A solução contempla mecanismos para registrar solicitações, acompanhar seu andamento, permitir sua análise e decisão e manter informações necessárias à rastreabilidade das operações. Dessa forma, busca-se reduzir a dependência de intervenções manuais no banco de dados e estabelecer um processo mais controlado e verificável.

Do ponto de vista técnico, o desenvolvimento envolveu conceitos de engenharia de software, aplicações web, persistência de dados relacionais, testes automatizados e conteinerização. Destacam-se a integração entre componentes de front-end e back-end, o uso de mecanismos de registro e recuperação do estado dos usuários e a manutenção de um histórico das operações. Esses fundamentos são apresentados no capítulo dedicado ao embasamento teórico.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um módulo de solicitações de alteração cadastral e de classificação no sistema Banco de Especialistas, contemplando os fluxos de criação, consulta, aprovação, rejeição e reversão, com mecanismos de rastreabilidade e registro das operações.

1.1.2 Objetivos específicos

- Levantar e analisar os requisitos e as regras de negócio relacionados às solicitações de alteração cadastral e de classificação dos especialistas.
- Modelar os componentes de persistência necessários ao registro das solicitações e à recuperação do estado dos usuários.
- Implementar as funcionalidades de gerenciamento das solicitações no back-end, incluindo as regras de negócio, a integração com o PostgreSQL e o acesso aos dados por meio do SQLAlchemy.
- Desenvolver as interfaces web necessárias à criação, ao acompanhamento e à análise das solicitações.
- Implementar mecanismos de rastreabilidade das operações, incluindo o registro de eventos e a apresentação de seu histórico.
- Desenvolver e executar testes unitários e de integração para verificar as regras de negócio e o funcionamento integrado dos componentes da aplicação.

1.2 Escopo e limitações

O trabalho abrange o desenvolvimento do módulo de solicitações e as funcionalidades necessárias ao gerenciamento das operações de alteração cadastral e de classificação dos especialistas no sistema Banco de Especialistas. Estão incluídos os mecanismos de registro, acompanhamento, decisão e reversão das solicitações, bem como os recursos de persistência e rastreabilidade associados a esse fluxo.

Não fazem parte do escopo a redefinição dos critérios de pontuação dos especialistas, a modificação do processo seletivo, a substituição do sistema de avaliação existente e a implementação de notificações por e-mail. Também não é contemplado o tratamento de múltiplas solicitações simultâneas para um mesmo especialista. Essas limitações delimitam as responsabilidades da solução desenvolvida e podem orientar evoluções futuras do sistema.

1.3 Procedimentos metodológicos

O trabalho caracteriza-se como um relato técnico de natureza aplicada e caráter descritivo, desenvolvido no contexto de um estágio supervisionado em desenvolvimento de software. Sua execução compreendeu o estudo da arquitetura e das regras de negócio do sistema, o levantamento e a análise dos requisitos, a modelagem dos componentes necessários, a implementação das funcionalidades e a realização de testes.

A verificação técnica envolveu testes unitários e de integração, destinados a avaliar o comportamento das funcionalidades implementadas e a interação entre os componentes da aplicação. Também foi verificado o funcionamento integrado do front-end e do back-end em ambiente de desenvolvimento conteinerizado com Docker. As atividades foram conduzidas no

contexto da arquitetura e das tecnologias já adotadas pelo projeto, considerando suas restrições e regras de negócio.

As atividades do módulo foram organizadas no OpenProject (OpenProject, 2026), com issues, responsáveis e prazos, o que apoiou a priorização das tarefas e o acompanhamento do progresso ao longo do estágio.

1.4 Contexto institucional

O estágio foi realizado no âmbito do Polo de Inovação do Instituto Federal da Paraíba, estrutura associada à Agência de Inovação da instituição (NEO-IFPB). O Polo atua no desenvolvimento de iniciativas de Pesquisa, Desenvolvimento e Inovação (PD&I), aproximando a instituição de demandas tecnológicas do setor produtivo e da sociedade.

As atividades técnicas foram desenvolvidas no Laboratório Assert, ambiente vinculado ao Polo de Inovação no qual são realizadas atividades de desenvolvimento e manutenção de sistemas computacionais para projetos institucionais. Nesse contexto, o estágio proporcionou a participação nas etapas de análise, implementação, integração e verificação de funcionalidades de uma aplicação web existente.

1.5 Organização do relatório

Além desta introdução, o relatório está organizado em três capítulos. O capítulo 2 apresenta os conceitos e as tecnologias que fundamentam o módulo. O capítulo 3 descreve as atividades do estágio, da análise dos requisitos à implementação e aos testes. O capítulo 4 reúne as considerações finais, com a síntese dos resultados, as limitações e as possibilidades de evolução do trabalho.

2 EMBASAMENTO TÉORICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

2.1 Arquitetura Web Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em provedores de serviços e requisitantes (TANENBAUM; van Steen, 2007). Nesse contexto, a comunicação orientada pelo estilo REST e pelo protocolo HTTP define interações sem estado, nas quais cada requisição é independente e deve carregar as informações necessárias ao seu processamento (FIELDING; NOTTINGHAM; RESCHKE, 2022).

Essa base teórica fundamenta a separação de responsabilidades do módulo: isolar a máquina de estados e as regras de persistência no servidor garante que a integridade dos dados e as transições de status da solicitação não dependam do cliente. Dessa forma, a aplicação no frontend atua exclusivamente na apresentação e na coleta de informações, consumindo a API REST para criar, acompanhar e analisar as requisições.

2.2 Flask

O Flask foi utilizado na API do módulo por permitir incorporar suas rotas à aplicação existente sem impor uma reorganização do sistema. Os *blueprints* agruparam as operações de criação, análise, acompanhamento e reversão das solicitações em um componente isolado, preservando as demais funcionalidades e facilitando a manutenção do módulo (PALLETS, 2024).

2.3 Persistência de dados com PostgreSQL e SQLAlchemy

O módulo utiliza PostgreSQL para armazenar solicitações, alterações cadastrais e registros de auditoria. A persistência dessas informações precisa manter o cadastro e o histórico coerentes, sobretudo quando uma decisão altera mais de uma entidade.

2.3.1 PostgreSQL

O PostgreSQL foi adequado por oferecer transações para as operações compostas do fluxo. Na aprovação, a mudança de categoria, a atualização da pontuação, a gravação do *snapshot* do estado anterior e a atualização do status da solicitação devem ser confirmadas em conjunto; se uma etapa falhar, nenhuma alteração deve permanecer aplicada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024).

2.3.2 SQLAlchemy

O SQLAlchemy foi adotado como ORM para coordenar as alterações de entidades relacionadas em uma única unidade de trabalho (BAYER, 2012). A sessão acumula as mudanças e, no *commit*, efetiva-as na transação; em caso de falha, o *rollback* descarta o conjunto. Assim, a aprovação não deixa a categoria atualizada sem o *snapshot* correspondente ou sem o novo status da solicitação.

2.4 Validação de Dados com Marshmallow e Flasgger

Na fronteira da API, os *schemas* do Marshmallow validam os campos e formatos recebidos antes que os dados cheguem às regras de negócio (MARSHMALLOW DEVELOPERS, 2024). Ao declarar os formatos esperados para as solicitações e suas respostas, também estabelecem o contrato consumido pelas interfaces de criação, acompanhamento e análise do módulo.

Complementarmente, o Flasgger foi utilizado na documentação e na especificação dos endpoints da API, permitindo descrever os parâmetros, os formatos esperados e as respostas das operações segundo as especificações OpenAPI/Swagger. Além da documentação interativa, a biblioteca oferece mecanismos de validação das requisições recebidas com base nos schemas definidos, desde que habilitados na configuração da aplicação. Dessa forma, sua utilização pode contribuir tanto para a explicitação do contrato da API quanto para a verificação da conformidade dos dados de entrada com as estruturas especificadas (FLASGGER DEVELOPERS, 2024).

2.5 Auditoria, Rastreabilidade e Versionamento de Dados

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

2.5.1 Auditoria e rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade

é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (FOWLER, 2004) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

2.5.2 Fluxo de aprovação com máquina de estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada e rejeitada) e de transições permitidas entre eles (SOMMERVILLE, 2011). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

2.5.3 Snapshot e reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (GAMMA et al., 1994), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (FOWLER, 2005; KLEPPMANN, 2017). O módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

2.5.4 Timeline e registro de eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

2.6 Verificação das Transições com Pytest

O Pytest foi empregado para verificar as regras de negócio do módulo, em especial as transições permitidas da máquina de estados: criação, aprovação, rejeição e reversão (HOLGER KREKEL AND PYTEST DEVELOPERS, 2024). Os testes também verificaram os efeitos dessas transições sobre a categoria, a pontuação e o histórico da solicitação.

2.7 Ambiente Reprodutível de Testes com Docker

O Docker foi utilizado para reproduzir o ambiente necessário aos testes de integração, incluindo a API e o PostgreSQL, de modo que a validação dos fluxos ocorresse sob uma configuração consistente entre execuções (DOCKER, 2024).

2.8 Controle de Versão e Gestão de Configuracao

A gestão de configuração de software compreende a identificação, o controle e o registro das alterações realizadas nos artefatos de um projeto ao longo do tempo, de modo que seja possível reconstituir versões anteriores e identificar a origem de cada mudança (SOMMERVILLE, 2011). O controle de versão é o mecanismo que concretiza esse registro sobre o código-fonte: cada alteração é capturada em um commit, com autor, data e descrição, e o histórico resultante permite recuperar estados anteriores do sistema.

O módulo utilizou Git (Git, 2026) para esse fim, com ramificações (branches) por funcionalidade, o que isolou o desenvolvimento do módulo das demais partes do Banco de Especialistas até que as alterações fossem verificadas. Os repositórios foram hospedados no GitLab (GitLab, 2026), adotado como padrão pelo Laboratório Assert para os projetos do Polo de Inovação, e nele foi configurada uma pipeline de integração contínua que executa a suíte de testes (Pytest) a cada envio, impedindo que alterações que quebrem as regras de negócio sejam incorporadas à base principal.

Cabe observar que o controle de versão atua sobre o código-fonte, e não sobre os dados da aplicação. A rastreabilidade das alterações cadastrais dos especialistas, tratada na seção 2.5, é um problema distinto, resolvido no domínio da aplicação por meio de solicitações, snapshots e registro de eventos.

2.9 Síntese do Capítulo

As tecnologias e os conceitos apresentados compõem a base sobre a qual o módulo de solicitações foi desenvolvido. A arquitetura cliente-servidor e o estilo REST definem a forma de comunicação entre as interfaces e a API; o Flask estrutura a aplicação, o SQLAlchemy e o PostgreSQL garantem a persistência transacional, e o Marshmallow assegura a validação e o contrato dos dados. Os conceitos de auditoria, máquina de estados, *snapshot* e registro de eventos sustentam os requisitos de rastreabilidade e reversão que motivaram o estágio. Por fim, o Pytest garante a verificação das regras de negócio, o Git organiza a colaboração e o Docker fornece o ambiente reprodutível de execução. O próximo capítulo descreve como esses elementos foram aplicados nas atividades realizadas.

3 ATIVIDADES REALIZADAS

Este capítulo apresenta as principais atividades realizadas no decorrer do estudo e do desenvolvimento da funcionalidade de alteração de categoria de especialistas. A partir da análise do sistema existente e do levantamento das regras de negócio, foi possível compreender o contexto institucional, os fluxos de operação e os requisitos necessários para a implementação de uma solução integrada, auditável e compatível com a arquitetura atual.

3.1 Estudo da Arquitetura do Sistema Existente

3.1.1 Visão Geral do Sistema

O primeiro passo para o desenvolvimento da funcionalidade consistiu na compreensão do sistema Banco de Especialistas, de sua finalidade institucional e de sua organização técnica. A plataforma permite o cadastramento e a classificação de profissionais interessados em participar de projetos de pesquisa, desenvolvimento e inovação vinculados ao IFPB.

Nesse contexto, foram estudadas as funcionalidades relacionadas ao cadastro dos especialistas, à organização por categorias e aos critérios de classificação utilizados pelo sistema. Também foram examinados os componentes da aplicação e suas interações, com o objetivo de identificar os pontos de integração necessários ao desenvolvimento do novo módulo.

3.1.2 Características do Código Existente e Decisão por Additive Change

A adoção do princípio de Additive Change foi uma decisão arquitetural relevante para garantir a compatibilidade com a base já existente. Esse princípio orienta a evolução do sistema por meio da adição de novas rotas, campos, serviços e módulos, sem a necessidade de reescrever estruturas centrais ou alterar de forma profunda o comportamento de componentes já consolidados.

Essa estratégia se mostrou adequada ao contexto do projeto, pois a aplicação legada foi estruturada em um padrão MVC com controladores responsáveis por múltiplas tarefas, incluindo validação, persistência, tratamento de exceções, registro de eventos e comunicação com camadas externas. Esse desenho favoreceu a rapidez inicial de implementação, mas também resultou em baixa coesão, acoplamento funcional e maior dificuldade de manutenção e evolução.

A partir dessa análise, a nova funcionalidade foi concebida como uma extensão incremental do sistema, preservando os comportamentos já consolidados e adicionando um fluxo específico para a solicitação, aprovação, auditoria e registro histórico das mudanças de categoria. Essa abordagem reduziu o risco de regressão e permitiu a inserção de novos mecanismos sem comprometimento da infraestrutura preexistente.

A seguir, é apresentado o padrão observado em controladores existentes, evidenciando as características que motivaram a escolha por uma evolução incremental.

Algoritmo 1: Padrão de implementação observado em controladores existentes.

```
@bp.route('', methods=['POST'])
@swag.validate('UserRegistration')
def register():
    if not validate_cpf(request.json['cpf']):
        raise ValidationError(details={'cpf': 'invalid'})

    user = User(**request.json)
    user.hash_password()

    try:
        db.session.add(user)
        db.session.commit()
    except IntegrityError as e:
        msg = e.args[0]
        if 'users_tipo_especialista_fkey' in msg:
            raise InvalidIdFieldError('specialist_type_id')
        elif 'users_cpf_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('cpf')
        elif 'users_email_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('email')
        else:
            raise e

    create_timeline_log(
        activity_type_id=ActivityTypeEnum.CADASTRADO,
        user_id=user.id)

    msg = 'Informe seus dados pessoais...'
    create_notification(title='Boas vindas!', message=msg, user=user)

    return UserSchema().dump(user), HTTPStatus.CREATED
```

O padrão exemplificado no Algoritmo 1 demonstra a forma como os controladores legados eram organizados. Em uma mesma função, eram concentradas validação de entrada, construção do objeto de domínio, hash de senha, persistência no banco de dados, tratamento de erros de integridade, registro de timeline, envio de notificação e serialização da resposta. A presença de documentação Swagger, mesmo quando omitida no trecho, reforça a concentração de responsabilidades no mesmo ponto de entrada da aplicação.

Esse desenho gera alguns acoplamentos relevantes. O primeiro deles está no tratamento de erro, em que a lógica depende de strings de constraint do banco, como `users_tipo_especialista_fkey` e `users_cpf_key`. Esse tipo de condição cria uma forte dependência com o esquema físico do banco de dados, de forma que qualquer renomeação ou alteração da constraint pode quebrar o fluxo de tratamento de exceções. O segundo acopla-

mento está na execução de operações colaterais após o commit, como a criação de registros de timeline e notificações. Nesse cenário, a criação do usuário e os eventos associados não são tratados como uma unidade transacional, de modo que falhas nas etapas posteriores podem deixar um registro principal persistido sem os dados complementares esperados.

A comparação com o novo módulo evidencia a importância da abordagem incremental. O desenvolvimento do módulo de solicitações foi organizado para separar responsabilidades e encapsular regras de negócio em serviços e camadas mais específicas, preservando o restante da aplicação e reduzindo o risco de efeitos colaterais.

Algoritmo 2: Estrutura do novo módulo, com separação de responsabilidades.

```
@bp_solicitacoes.route('', methods=['POST'])
@schema_required(SolicitacaoCreateSchema)
def criar_solicitacao(payload):
    solicitacao = solicitacao_service.criar(
        solicitante_id=current_user.id,
        tipo=payload['tipo'],
        dados=payload['dados'],
    )
    return SolicitacaoSchema().dump(solicitacao), HTTPStatus.CREATED


def criar(solicitante_id, tipo, dados):
    campos = SCHEMA_POR_TIPO[tipo].load(dados)
    solicitacao = Solicitacao(...)

    with db.session.begin_nested():
        db.session.add(solicitacao)
        registrar_evento(solicitacao, Evento.CRIADA)

    return solicitacao
```

O Algoritmo 2 ilustra o padrão adotado no novo módulo. A validação ocorre na fronteira, por meio do decorador `@schema_required(SolicitacaoCreateSchema)`, de modo que o serviço recebe um payload já validado. O controlador limita-se a delegar a execução ao serviço e serializar a resposta, sem concentrar hash, tratamento de erro do banco ou envio de notificações. A seleção do schema por tipo, por meio de `SCHEMA_POR_TIPO[tipo].load(dados)`, representa o mecanismo das restrições dinâmicas descritas na análise de regras de negócio e reforça a separação entre validação e persistência. Por fim, a persistência e o registro do evento são agrupados em uma mesma unidade de trabalho, eliminando o intervalo entre o commit e os efeitos colaterais observado no Algoritmo 1.

A escolha por Additive Change permitiu incorporar o novo fluxo sem reescrever os controladores existentes, preservando o comportamento consolidado da aplicação.

3.1.3 Análise das Regras de Negócio

O sistema possui um modelo de regras de negócio relacionadas à classificação de especialistas e à alteração de seus dados cadastrais. Condições essas que deveriam ser respeitadas pela nova funcionalidade, bem como os efeitos das operações sobre os registros existentes.

Foram examinados os critérios relacionados à mudança de categoria, à atualização da pontuação e às responsabilidades dos usuários envolvidos no processo. Também foi necessário distinguir as operações de solicitação das ações de aprovação e rejeição, uma vez que essas etapas possuem finalidades e consequências distintas. O levantamento dessas regras serviu de base para definir o comportamento esperado do módulo e evitar que as novas operações comprometessem as restrições já estabelecidas pelo sistema.

Com base no sistema atual, os principais desafios para realizar a transição de categoria de um especialista começam pelo fato de que a categoria de um especialista não estar armazenada no modelo Specialist, mas sim no User.

3.1.4 Entendimento do Fluxo Antigo e Diagnóstico de Demanda

Antes do desenvolvimento do módulo, as solicitações de mudanças de categoria e de atualização cadastral eram realizadas exclusivamente por e-mail. Após o recebimento do pedido, a alteração dependia da intervenção de um técnico, que executava manualmente as operações diretamente na base de dados por meio de scripts SQL reutilizáveis.

Esse procedimento estabelecia uma forte dependência entre a solicitação do usuário e a disponibilidade de um responsável com acesso privilegiado ao banco de dados. Além disso, o acompanhamento dos pedidos, a validação das justificativas e o histórico das alterações ficavam dispersos nas caixas de entrada de e-mail, fora de um fluxo centralizado e auditável.

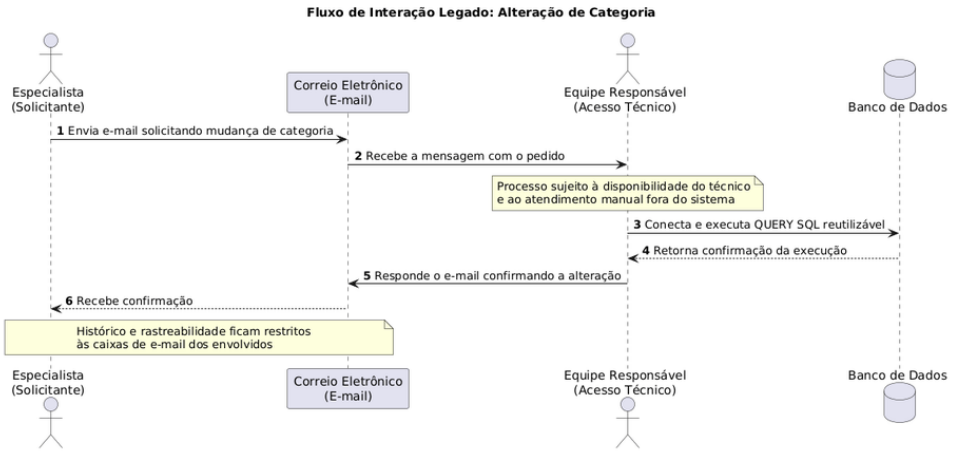


Figura 1 – Fluxo de alteração de categoria de especialistas antes do desenvolvimento do módulo.

Fonte: Elaboração própria (2026).

Para dimensionar a recorrência desse gargalo operacional, realizou-se o levantamento

dos chamados recebidos pelo e-mail de suporte no período de 30 de maio de 2025 a 4 de agosto de 2026. No total, foram catalogados 141 chamados de suporte diretamente vinculados a solicitações manuais de ajuste cadastral e de pontuação.

A Tabela 1 apresenta a distribuição das solicitações conforme o tipo de vínculo do usuário no sistema.

Tabela 1 – Volume de solicitações de suporte por tipo de inscrição (maio/2025 a agosto/2026).

Tipo de Inscrição	**Quantidade de Chamados**	**Porcentagem (%)**
Colaborador(a) externo	79	56,0%
Estudante	47	33,3%
Servidor(a)	15	10,6%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

A Tabela 2 detalha os motivos apresentados nesses atendimentos.

Tabela 2 – Distribuição das solicitações de suporte por categoria de problema.

Categoria do Problema	**Quantidade**	**Porcentagem (%)**
Outro (especificar)	59	41,8%
Completar cadastro (dados pessoais)	34	24,1%
Cadastrar pontuação	34	24,1%
Cadastrar no sistema	7	5,0%
Fazer login no sistema	7	5,0%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

A análise empírica revela que as demandas por atualização de cadastro e inclusão de pontuação somam juntas 48,2% de todos os chamados mapeados. Quando somadas às demandas genéricas ("Outro"), constata-se que a grande maioria das interações via e-mail tratava-se de intervenções manuais que poderiam ser automatizadas.

A execução de queries manuais em ambiente de produção para atender a esse volume constante de chamados apresentava riscos críticos:

- **Vulnerabilidade de segurança:** risco de erro humano na execução de scripts diretos no SGBD;
- **Falta de rastreabilidade:** ausência de registros sobre quem aprovou ou alterou determinado dado;
- **Ineficiência operacional:** alocação de esforço técnico em tarefas repetitivas de suporte administrativo;
- **Atrasos no atendimento:** dependência da disponibilidade da equipe de TI para demandas administrativas do sistema.

O diagnóstico comprovou a necessidade de aposentar o fluxo manual e implementar um módulo integrado ao sistema, capaz de formalizar solicitações, gerenciar aprovações via máquina de estados e manter um histórico auditável das alterações.

3.1.5 Levantamento de Requisitos

O levantamento de requisitos teve como objetivo definir as funcionalidades e as propriedades de qualidade necessárias à integração do módulo de solicitações ao sistema Banco de Especialistas. A solução foi concebida para preservar o comportamento das funcionalidades existentes e formalizar o processo de alteração de categoria e de dados cadastrais dos especialistas, mediante validação das entradas, controle de aprovação e registro das operações realizadas.

Os requisitos funcionais especificam as operações disponibilizadas pelo módulo e seus efeitos sobre os registros do sistema. Os requisitos não funcionais estabelecem as condições de integridade, segurança, rastreabilidade, compatibilidade e manutenção que orientam a implementação da solução.

3.1.5.1 Requisitos Funcionais

Os requisitos funcionais abrangem a criação e o processamento de solicitações, a consulta e o acompanhamento das demandas, a aplicação das alterações aprovadas, a preservação do estado anterior dos registros e o controle de acesso às operações. As decisões e as alterações devem ser executadas exclusivamente por usuários autorizados, conforme as regras de negócio estabelecidas.

Quadro 1 – Requisitos funcionais do módulo de solicitações.

ID	Descrição	Tipo de solicitação	Origem	Implementação
RF01	Permitir a criação de solicitações de mudança de categoria, com indicação da categoria de destino e justificativa obrigatória.	Categoria	Seção 3.1.4	Seção 3.2
RF02	Permitir a criação de solicitações de atualização de pontuação, com indicação dos itens e das respectivas comprovações.	Pontuação	Seção 3.1.4	Seção 3.2
RF03	Permitir a criação de solicitações de complemento cadastral ou de recurso, com descrição do problema identificado.	Complemento	Seção 3.1.4	Seção 3.2
RF04	Permitir a consulta das solicitações, com filtros por tipo e por situação, conforme o perfil do usuário (solicitante ou avaliador).	Todos	Seção 3.1.4	Seção **??**
RF05	Permitir a aprovação ou rejeição de solicitações, registrando o responsável e a decisão tomada.	Todos	Seção 3.1.4	Seção 3.2
RF06	Aplicar, após a aprovação, as alterações correspondentes à categoria de destino do especialista.	Categoria	Seção 3.1.3	Seção 3.2
RF07	Aplicar, após a aprovação, a atualização da pontuação do especialista correspondente aos itens aprovados.	Pontuação	Seção 3.1.3	Seção 3.2
RF08	Registrar o estado anterior do especialista antes da aplicação de alterações registradas em snapshot.	Categoria	Seção 3.1.4	Seção 3.3
RF09	Permitir a reversão de alterações aprovadas, restaurando o estado anterior registrado.	Categoria	Seção 3.1.4	Seção 3.3
RF10	Registrar e disponibilizar uma linha do tempo cronológica dos eventos associados a cada solicitação.	Todos	Seção 3.1.3	Seção 3.3
RF11	Restringir as operações de criação, consulta e decisão a usuários autenticados e autorizados, conforme o perfil de acesso.	Todos	Seção 3.1.4	Seção 3.2

Fonte: Elaboração própria (2026). **Nota:** O requisito RF03 refere-se à abertura de solicitações de complemento cadastral ou recurso. Após a aprovação ou rejeição da demanda, o sistema registra a decisão e o encaminhamento correspondente, sem alterar automaticamente o perfil do especialista.

3.1.5.2 Requisitos Não Funcionais

Os requisitos não funcionais estabelecem as propriedades de qualidade necessárias ao funcionamento do módulo e à sua integração ao sistema existente. Esses requisitos abrangem a integridade das operações, a rastreabilidade das alterações, a segurança do controle de acesso, a validação dos dados, a compatibilidade com as funcionalidades existentes, a extensibilidade da solução, a testabilidade das regras de negócio e a reprodutibilidade do ambiente de testes.

Quadro 2 – Requisitos não funcionais do módulo de solicitações.

ID	**Categoria**	**Descrição**	**Mecanismo de atendimento**
RNF01	Integridade	As alterações decorrentes da aprovação devem ser executadas atomicamente, evitando a persistência parcial dos registros envolvidos na operação.	Transações PostgreSQL e SQLAlchemy
RNF02	Rastreabilidade	As informações de auditoria devem ser preservadas de forma que seja possível reconstituir, a qualquer momento, o histórico de decisões e o estado anterior das alterações registradas em snapshot.	Entidade Snapshot e linha do tempo de eventos
RNF03	Segurança	O controle de acesso deve ser aplicado no servidor, de modo que as operações não possam ser executadas por chamadas diretas à API que contornem a interface.	Verificação de permissões nas rotas da API
RNF04	Validação	Os dados recebidos pela API devem ser validados no servidor, de modo que a validação do servidor seja autoritativa, independentemente das verificações realizadas pela interface.	Schemas Marshmallow
RNF05	Compatibilidade	A incorporação do módulo deve preservar o comportamento das funcionalidades existentes não abrangidas pelo módulo.	Blueprints Flask e estratégia de mudança aditiva (*Additive Change*)
RNF06	Extensibilidade	A inclusão de novos tipos de solicitação deve permitir o reaproveitamento do fluxo principal, limitando as alterações necessárias às regras específicas de cada tipo.	Formulário único e configuração `SCHEMA_POR_TIPO`
RNF07	Testabilidade	As regras de negócio devem permitir testes isolados sem requisição HTTP, de modo que possam ser verificadas independentemente da camada de comunicação.	Separação da camada de serviço e Pytest
RNF08	Reprodutibilidade	O ambiente de testes deve poder ser configurado e executado de maneira reproduzível, com dependências e procedimentos documentados.	Docker e Docker Compose

Fonte: Elaboração própria (2026).

3.2 Desenvolvimento do Módulo de Solicitações

3.2.1 Endpoints

A camada de rotas do módulo de solicitações foi implementada como um *Blueprint* do Flask, registrado sob o prefixo `/solicitations`. A escolha do *Blueprint* decorre diretamente da estratégia de *Additive Change* descrita na Seção **??**: o módulo é incorporado como um componente isolado, sem alterar os controladores existentes. As rotas atuam como camada de fronteira — recebem a requisição, aplicam o controle de acesso, delegam o processamento ao `SolicitationService` e serializam a resposta —, mantendo fora do controlador as regras de negócio, a persistência e o tratamento de exceções de domínio. A documentação Swagger foi externalizada em arquivos YAML, referenciados pelo decorador `@swag_from`, em contraste com o padrão observado nos controladores legados (Algoritmo 1), nos quais a documentação ocupava a maior parte da função.

O Quadro 3 relaciona os endpoints implementados, a função responsável por cada um, a restrição de acesso aplicada e os requisitos funcionais atendidos.

Quadro 3 – Endpoints implementados no módulo de solicitações.

Método	**Rota**	**Função**	**Controle de acesso**	**Requisitos**
POST	/	`save_solicitation`	Usuário autenticado	RF01, RF02, RF03
GET	/	`get_solicitations_page`	Autenticado com perfil de visualização	RF04
GET	/me	`get_my_solicitation`	Usuário autenticado (restrito ao próprio usuário)	RF04
GET	/<id>	`get_detail_solicitation`	Autenticado, com verificação de acesso no serviço	RF04, RF11

Fonte: Elaboração própria (2026). Obs: usando com báse que todas as rotas são prefixadas com `/solicitations`.

A rota de criação (`POST /solicitations`) concentra o padrão adotado nas demais. O controlador limita-se a extrair os dados da requisição — aceitando tanto `multipart/form-data`, quando há arquivo anexado, quanto JSON, quando não há — e a delegar o processamento ao serviço, conforme ilustrado no Algoritmo 3.

Quatro características do padrão merecem destaque. A primeira é o **controlador fino**: a função não constrói entidades, não valida campos individualmente, não executa consultas e não trata exceções de banco — responsabilidades que, no padrão legado (Algoritmo 1), estavam concentradas na própria rota. A segunda é o **controle de acesso declarativo**, aplicado pelo decorador `@auth.login_required`, que aceita um parâmetro opcional de perfil; a rota de listagem geral exige perfil de visualização, enquanto a rota `/me` restringe o resultado ao próprio usuário autenticado, independentemente do perfil. A terceira é a **validação na fronteira** por schemas Marshmallow: a consulta é interpretada por `SolicitationQuerySchema`, e a

Algoritmo 3: Estrutura da rota de criação de solicitação.

```
@bp.route("", methods=["POST"])
@auth.login_required
@swag_from("docs/create_solicitation.yml", validation=False)
def save_solicitation():
    form_data = request.form.to_dict() if request.form else None
    json_data = request.get_json(silent=True) or {}
    file = request.files.get("document_file")

    return SolicitationService.create_or_update(
        user_id=g.user.id,
        form_data=form_data,
        file=file,
    )
```

resposta é serializada por `SolicitationSchema` ou `PaginatedResponseSchema`, de modo que o formato de entrada e o de saída não dependem da interface que consome a API, atendendo ao RNF04. A quarta é a **externalização da documentação**: o decorador `@swag_from` referencia um arquivo YAML por rota, com `validation=False` indicando que a validação é responsabilidade dos schemas e não do Swagger.

A listagem foi implementada por meio de uma função auxiliar, `_fetch_solicitations`, que centraliza a interpretação dos parâmetros de paginação, ordenação e filtro (situação, intervalo de datas e usuário). O endpoint `/solicitations/me` reutiliza a mesma função forçando o identificador do usuário autenticado, enquanto o endpoint `/solicitations` permite filtro opcional por usuário, desde que o solicitante possua o perfil de visualização. Essa centralização evita a duplicação da lógica de consulta entre rotas, que, no padrão legado, costumava ser reproduzida em cada controlador.

Do ponto de vista dos requisitos, os endpoints concentram os RF01 a RF04 (criação e consulta) e o RF11 (restrição de operações a usuários autorizados). As operações de decisão, aplicação de efeitos, snapshot e reversão, correspondentes aos RF05 a RF09, são implementadas por rotas específicas descritas na Seção 3.2.

3.2.2 Integração com o Banco de Dados

A persistência do módulo foi implementada com SQLAlchemy sobre PostgreSQL, seguindo o padrão já adotado pelo sistema. O mapeamento foi feito em estilo declarativo, com uma tabela principal (`solicitations`) e duas tabelas de referência (`solicitation_status` e `solicitation_category`), que funcionam como *lookup tables* para os estados e os tipos de solicitação. A escolha de manter tabelas de referência, em vez de armazenar os valores como texto livre, garante integridade referencial no banco e permite a evolução do domínio sem migrations estruturais a cada novo valor. O Quadro 4 descreve as tabelas do módulo, suas

colunas e restrições.

Quadro 4 – Modelo de dados do módulo de solicitações.

Tabela	**Coluna**	**Tipo / Restrição**	**Observação**
`solicitations`	`id`	Inteiro, PK	Identificador da solicitação
	`id_user`	FK → `users`, obrigatório	Solicitante
	`id_status`	FK → `solicitation_status`, padrão `CREATED`	Estado atual
	`id_category`	FK → `solicitation_category`, opcional	Tipo de solicitação; nulo em rascunho
	`specialist_type_id`	FK → `users_specialist_types`, opcional	Categoria de destino
	`title`	Texto(128), obrigatório	Título
	`description`	Texto(640), obrigatório	Justificativa ou descrição
	`file_path`	Texto(256), opcional	Caminho do documento anexado
	`created_at`	DataHora, padrão `agora()`	Criação
	`last_update`	DataHora, atualizado em alteração	Última modificação
`solicitation status`	`id`, `name`	PK, Texto(20) único	`CREATED`, `PENDING`, `APPROVED`, `REJECTED`
`solicitation category`	`id`, `name`	PK, Texto(20) único	`CHANGE_CATEGORY`, `RESOURCE`, `UPDATE_POINTS`
`solicitation reviews`	`id_solicitation`	FK → `solicitations`, cascata	Decisões e pareceres das análises

Fonte: Elaboração própria (2026).

O Algoritmo 4 descreve, em pseudocódigo, a estrutura da entidade principal, com seus atributos, restrições, índices e relacionamentos, e as enumerações que definem os estados e os tipos de solicitação.

A decisão de modelagem central é a **abordagem polimórfica em tabela única**: os três tipos de solicitação (`CHANGE_CATEGORY`, `RESOURCE` e `UPDATE_POINTS`) compartilham a mesma tabela `solicitations`, discriminados pelo campo `id_category`. Essa escolha evita a

Algoritmo 4: Estrutura da entidade `Solicitation` e enumerações de domínio (pseudocódigo).

```
ENUMERAÇÃO SolicitationStatus
    CREATED  = 1    // rascunho, em elaboração
    PENDING  = 2    // submetida, aguardando análise
    APPROVED = 3    // deferida
    REJECTED = 4    // indeferida

ENUMERAÇÃO SolicitationCategory
    CHANGE_CATEGORY = 1    // alteração de categoria
    RESOURCE        = 2    // recurso / complemento cadastral
    UPDATE_POINTS   = 3    // atualização de pontuação

ENTIDADE Solicitation
    TABELA: solicitations

    ATRIBUTOS
        id                : Inteiro, Chave Primária
        id_user           : Inteiro, FK -> User, obrigatório
        id_status         : Inteiro, FK -> SolicitationStatus,
                            padrão CREATED
        id_category       : Inteiro, FK -> SolicitationCategory,
                            opcional (discriminador de tipo)
        specialist_type_id : Inteiro, FK -> UserSpecialistType,
                            opcional (categoria de destino)
        title             : Texto(128), obrigatório
        description       : Texto(640), obrigatório
        file_path         : Texto(256), opcional
        created_at        : DataHora, padrão agora()
        last_update       : DataHora, atualizado em cada alteração

    RELACIONAMENTOS
        user    : User                 (muitos-para-um)
        reviews : Lista<Review>        (um-para-muitos, cascata)

    ÍNDICES
        idx_solicitation_user     -> (id_user)
        idx_solicitation_status   -> (id_status)
        idx_solicitation_created  -> (created_at)
```

fragmentação em três tabelas com colunas comuns replicadas (título, descrição, autor, estado, timestamps) e concentra a lógica de listagem, filtragem e paginação em uma única consulta, independentemente do tipo. Campos específicos de um tipo, como `specialist_type_id`, são opcionais e populados apenas quando aplicáveis: no fluxo de alteração de categoria, o campo indica a categoria de destino; nos fluxos de recurso e de atualização de pontuação, permanece nulo. A abordagem foi adotada em vez da herança de tabela por subtipo (*joined table inheritance*) por não haver, no escopo do módulo, atributos exclusivos de um tipo que justificassem tabelas próprias.

O **estado de rascunho** é consequência direta dessa modelagem. O estado `CREATED` representa uma solicitação em elaboração, que pode ser salva e editada antes da submissão. Nesse estado, o campo `id_category` pode ser nulo, permitindo que o usuário salve a solicitação parcialmente preenchida e escolha o tipo mais adiante. A submissão para análise corresponde à transição de `CREATED` para `PENDING`, momento a partir do qual o tipo passa a ser obrigatório na prática, ainda que a restrição não esteja expressa no esquema. A alternativa — tornar `id_category` obrigatório e criar uma tabela separada para rascunhos — foi descartada por duplicar a estrutura e a lógica de consulta sem ganho proporcional.

A **dupla representação de estados e categorias** — enumeração em Python e tabela de referência no banco — atende a duas necessidades distintas. A enumeração serve à aplicação, garantindo, em tempo de desenvolvimento, que apenas valores conhecidos sejam usados e permitindo comparações explícitas no código. A tabela de referência serve ao banco, garantindo integridade referencial e viabilizando junções e filtros eficientes nas consultas de listagem. O custo é a disciplina de manter os dois em sincronia a cada novo valor; o benefício é que o estado de uma solicitação nunca fica armazenado como texto livre, o que facilitaria inconsistências e inviabilizaria restrições declarativas.

A **chave estrangeira para** `users`, e não para a entidade de especialista, reflete no esquema o desafio identificado na Seção 3.1.3: a categoria do especialista está em `users`, e não em `specialists`. A solicitação referencia o usuário e, quando aplicável, indica a categoria de destino em `specialist_type_id`, sem duplicar a classificação corrente. Isso mantém o modelo coerente com o restante do sistema e evita que a solicitação armazene estados intermediários que precisariam ser sincronizados com a tabela de origem.

A **indexação** foi definida a partir das consultas efetivamente implementadas nas rotas (Seção 3.2.1): listagem por usuário (`idx_solicitation_user`), listagem por situação (`idx_solicitation_status`) e ordenação por data de criação (`idx_solicitation_created`). Consultas por identificador já são otimizadas pela chave primária e não exigem índice adicional. A escolha privilegia os caminhos de leitura da interface — listagem do solicitante, listagem do avaliador e ordenação cronológica — em vez de uma indexação genérica e não fundamentada.

O **armazenamento do arquivo** foi delegado ao sistema de arquivos: o atributo `file_path` guarda o caminho do documento anexado, não o conteúdo binário. Essa decisão mantém o banco fora do caminho de operações de I/O de arquivos, que são executadas fora da

transação principal, e evita o inchaço da tabela. A referência ao arquivo permanece no mesmo registro da solicitação, mantendo a consistência entre o documento e os metadados que o descrevem.

Por fim, o **controle temporal** foi delegado ao banco: `created_at` recebe o instante corrente na inserção e `last_update` é atualizado automaticamente a cada alteração, de modo que o momento da última modificação é registrado mesmo em operações que não passem pela aplicação. Isso atende parcialmente ao RNF02, no aspecto temporal da rastreabilidade; a dimensão semântica — quem alterou, de qual estado para qual estado, com qual justificativa — é atendida pelo mecanismo de snapshot e pela linha do tempo, descritos na Seção 3.3. A atomicidade das operações (RNF01), por sua vez, não é garantida no modelo, mas na camada de serviço, por meio da unidade de trabalho do SQLAlchemy (`session`): o modelo define as restrições; a transação define o momento em que elas são verificadas em conjunto.

3.2.3 Autorização e Controle de Acesso

A autorização foi implementada considerando os diferentes papéis envolvidos no fluxo, de modo que apenas usuários com permissão adequada possam aprovar, rejeitar ou alterar as informações relacionadas à categoria do especialista. Esse controle é essencial para preservar as regras de negócio e evitar ações indevidas no sistema.

3.2.4 Aprovação e Rejeição

A funcionalidade de aprovação e rejeição foi estruturada como etapas distintas, cada uma com consequências específicas sobre o estado da solicitação. A aprovação formaliza a decisão e desencadeia a mudança de categoria, enquanto a rejeição mantém o registro do pedido sem alteração do estado do especialista.

3.2.5 Mudança de categoria

A mudança de categoria representa o ponto central do módulo, sendo executada apenas quando a solicitação é aprovada e todas as regras pertinentes são satisfeitas. Esse processo foi implementado de forma a preservar a integridade dos dados e a lógica do sistema, evitando alterações abruptas ou inconsistentes em cadastros existentes.

3.2.6 Atualização de pontuação

A atualização de pontuação foi tratada como parte do processo de alteração de categoria, considerando os critérios de classificação previamente definidos. Essa etapa garante que a nova categoria reflita corretamente o histórico e os dados atualizados do especialista, mantendo a coerência da classificação institucional.

3.3 Implementação do Sistema de Snapshot e Auditoria

3.3.1 Entidade Snapshot e Modelo de Persistência

A entidade `SpecialistSnapshot` foi criada para preservar o estado do especialista antes da aplicação de alterações aprovadas no fluxo de mudança de categoria. O nome reflete o escopo: o snapshot cobre o especialista, e é acionado exclusivamente pelo tipo `CHANGE_CATEGORY` — recurso e atualização de pontuação não geram snapshot, pelas razões discutidas na Seção **??**. O Quadro 5 descreve sua estrutura.

Quadro 5 – Modelo da entidade `SpecialistSnapshot`.

Coluna	**Tipo**	**Restrição**	**Observação**
`id`	Inteiro	Chave primária	Identificador do snapshot
`specialist_id`	Inteiro	Obrigatório, indexado	Especialista cujo estado foi capturado; sem FK
`snapshot`	JSONB	Obrigatório	Estado anterior serializado
`authorizer_id`	Inteiro	Obrigatório	Usuário que autorizou a alteração; sem FK
`created_at`	DataHora	Padrão `agora()`	Momento da captura

Fonte: Elaboração própria (2026).

Duas decisões de modelagem merecem destaque. A primeira é a **ausência de chave estrangeira** em `specialist_id` e `authorizer_id`. Trata-se de uma escolha deliberada para uma entidade de auditoria: o snapshot é um registro histórico, e sua integridade não deve depender da existência atual da entidade referenciada. Se o especialista ou o autor da decisão for removido do sistema, o snapshot permanece consultável, preservando a memória do que ocorreu. A contrapartida é a possibilidade de referências órfãs e a ausência de integridade referencial garantida pelo banco — custo aceito em troca da preservação histórica. A alternativa — FK com `ON DELETE SET NULL` — destruiria justamente a informação que o snapshot existe para preservar.

A segunda é a **ausência de vínculo explícito com a solicitação** que originou o snapshot. A entidade referencia o especialista, não a solicitação nem a revisão. Essa decisão tem consequência direta sobre a reversão (Seção 3.3.2): a operação de reversão busca o snapshot mais recente do especialista, e não o snapshot "daquela solicitação". Para o fluxo atual — em que se reverte a última alteração aprovada —, a modelagem é suficiente. Para reversão de uma alteração específica no meio do histórico, seria necessário vincular o snapshot à solicitação ou à revisão correspondente, o que está registrado como limitação em Seção 3.3.4.

O campo `snapshot` armazena o estado anterior em JSONB. A escolha do formato, em vez de colunas relacionais, decorre de três fatores. O primeiro é a **coesão do estado**: os campos capturados descrevem um único estado lógico do especialista e fazem sentido juntos; armazená-los em uma coluna única preserva essa coesão e evita reconstituição por junção.

O segundo é a **evolução do esquema**: se o conjunto de campos a capturar for ampliado, um documento JSONB acomoda novos campos sem migration; colunas adicionais exigiriam alteração de esquema e migração de dados históricos. O terceiro é a **legibilidade**: consultas diretas ao documento permitem visualizar o estado anterior sem conhecer o esquema das tabelas de origem.

As contrapartidas também precisam ser declaradas. O JSONB não impõe integridade referencial interna — o documento é um retrato, não uma referência viva, e não há como garantir que os campos correspondam ao esquema atual do especialista depois de migrations futuras. O formato não é validado pelo banco: a estrutura do documento depende da aplicação que o grava. Essas limitações são aceitáveis no escopo do módulo, porque o snapshot é um registro histórico e **deve** preservar o estado como era no momento da captura, e não acompanhar a evolução do esquema. A alternativa — colunas relacionais com FK para as tabelas de origem — traria integridade, mas produziria snapshots que se corrompem quando a tabela de origem muda, anulando o propósito de preservar o estado anterior.

A captura é feita antes da aplicação dos efeitos e dentro da mesma unidade de trabalho da aprovação (Seção **??**): o snapshot, a mudança de categoria e o registro da revisão são confirmados em conjunto. Se qualquer etapa falhar, o snapshot é descartado junto, garantindo que ele exista se e somente se a alteração correspondente foi efetivada. Isso atende ao RNF01 e sustenta o RNF02: sem atomicidade, um snapshot poderia registrar um estado anterior que nunca foi alterado, ou uma alteração sem snapshot correspondente.

O campo `authorizer_id` registra o usuário responsável pela autorização da alteração, separadamente do autor da revisão. Na prática, os dois coincidem no fluxo atual — o avaliador que aprova a revisão é o mesmo que autoriza a alteração —, mas manter o campo explícito na entidade permite que, no futuro, a autorização seja delegada ou automatizada sem alteração do modelo. A separação também garante que a informação esteja presente na própria entidade de auditoria, e não apenas derivável por junção com a revisão.

3.3.2 Reversão de Alterações

A reversão permite desfazer uma alteração de categoria aprovada, restaurando o estado do especialista registrado no snapshot mais recente. A operação é acionada por rota administrativa e está sujeita a controle de acesso, como as demais operações de decisão (RNF03).

A reversão não é uma operação destrutiva: o snapshot não é apagado após o uso, e a reversão é registrada como evento próprio na linha do tempo (Seção 3.3.3). O histórico completo é preservado — a solicitação original, a revisão que aprovou, o snapshot capturado, a reversão e o estado resultante. Essa é a diferença entre reversão e exclusão de histórico: a primeira adiciona um evento, a segunda remove o passado. A escolha pela primeira é o que sustenta a auditoria.

O Algoritmo 5 descreve o fluxo em pseudocódigo, com base na recuperação do snapshot

mais recente do especialista.

Algoritmo 5: Fluxo de reversão (pseudocódigo).

```
FUNÇÃO reverter(especialista_id, autorizador):
    especialista = buscar Specialist por id
    SE não existe:
        erro "especialista não encontrado"

    snapshot = buscar SpecialistSnapshot
               filtrando specialist_id = especialista.id
               ordenando por created_at DESC
               limitando a 1 resultado
    SE não existe:
        erro "nenhum snapshot encontrado"

    estado_anterior = desserializar(snapshot.snapshot)

    EM UMA TRANSAÇÃO:
        restaurar(especialista, estado_anterior)
        registrar_evento(especialista, REVERTIDA,
                         autorizador, data)

    RETORNAR confirmação
```

Quatro decisões delimitam o escopo da reversão. A primeira é que ela restaura o **snapshot mais recente**, não o snapshot de uma solicitação específica. Isso decorre da modelagem da entidade (Seção 3.3.1), que referencia o especialista sem vincular-se à solicitação. A consequência é que a reversão desfaz a última alteração aprovada, e não uma alteração qualquer no histórico. Para o caso de uso original — corrigir uma mudança de categoria recente que se mostrou inadequada —, a semântica é adequada. Para reverter uma alteração específica no meio do histórico, seria necessário alterar o modelo para vincular o snapshot à solicitação. A limitação está registrada em Seção 3.3.4.

A segunda é que a reversão restaura apenas os campos capturados no snapshot, sem desfazer alterações feitas ao especialista por outros fluxos no intervalo. Se, após a aprovação, o especialista atualizou dados pessoais ou pontuação por outro caminho, esses dados não são afetados. A escolha é conservadora e evita que uma reversão de categoria apague, por tabela, alterações não relacionadas.

A terceira é que a reversão **não altera o estado da revisão** que aprovou a solicitação. A revisão permanece registrada como aprovada, e a reversão é um evento subsequente — o que preserva o histórico de decisões.

A quarta é que a operação adiciona um novo evento à linha do tempo, mas não gera um novo snapshot. Isso poderia ser questionado — se a reversão também é uma alteração, por que não capturar o estado anterior a ela? A resposta é que reverter uma reversão seria

equivalente a reaplicar a alteração original, e isso é feito criando-se uma nova solicitação, não reusando o snapshot antigo. A decisão mantém o modelo simples: um snapshot por alteração aprovada, um evento de reversão por reversão executada.

3.3.3 Linha do Tempo de Eventos

A linha do tempo é a apresentação cronológica dos eventos associados a um especialista: criação da solicitação, submissão para análise, decisão da revisão, captura de snapshot e, quando aplicável, reversão. A implementação **não mantém tabela própria de eventos**: a linha do tempo é derivada das entidades que já registram cada evento com autor e data — `Solicitation` (`created_at`), `SolicitationReview` (autor, veredicto, comentário, data) e `SpecialistSnapshot` (autorização, captura, data). A escolha por derivação, em vez de uma tabela de log genérica, evita duplicação de informação e garante que a linha do tempo nunca divirja das entidades de origem: se a revisão registrou uma decisão, ela aparece na linha do tempo; se o snapshot foi capturado, ele também aparece.

O preço da derivação é que a consulta precisa agregar três fontes e ordená-las por data. Isso é feito no serviço de consulta da linha do tempo, que retorna a lista consolidada para a interface. A alternativa — tabela de eventos dedicada, alimentada por gatilhos — teria a vantagem de uma consulta única, mas exigiria sincronização com as entidades de origem e abriria margem para inconsistência. Para o volume atual, a derivação é a solução mais simples e mais robusta.

A linha do tempo cumpre duas funções. Para o solicitante, mostra o andamento da solicitação e a justificativa de cada decisão. Para o avaliador e para auditoria, permite reconstituir o processo completo, com autores, datas e estados intermediários. Em ambos os casos, a informação apresentada é a mesma que sustenta a rastreabilidade — a interface apenas a torna legível.

3.3.4 Rastreabilidade e Limitações

O conjunto formado por solicitação, revisão, snapshot e linha do tempo atende ao requisito de rastreabilidade (RNF02) em três dimensões. A dimensão **temporal** é coberta pelos carimbos automáticos das entidades. A dimensão **autoral** é coberta pelo registro do solicitante na solicitação, do avaliador na revisão e do autorizador no snapshot. A dimensão **semântica** — o que mudou e por quê — é coberta pelo snapshot (estado anterior) e pela revisão (justificativa). As três juntas permitem reconstituir, para qualquer solicitação, quem pediu, quem decidiu, quando, com qual justificativa e de qual estado para qual estado.

O mecanismo tem limitações que precisam ser declaradas. A primeira é a já discutida **reversão pelo snapshot mais recente**: como o snapshot não referencia a solicitação, a reversão desfaz a última alteração aprovada, e não uma alteração específica. A segunda é a **ausência de exclusão lógica**: o `cascade="all, delete-orphan"` na relação entre solicitação e revisões implica que a remoção física de uma solicitação apaga o histórico

associado. O módulo não expõe rota de exclusão, de modo que o cascade atua apenas como salvaguarda de integridade, mas a ausência de uma política de retenção é uma lacuna que pode ser endereçada em trabalho futuro. A terceira é a **ausência de vínculo do snapshot com a solicitação**: o snapshot referencia o especialista, não a solicitação, o que simplifica a modelagem e atende ao caso de uso atual, mas impede auditoria retroativa que relacione snapshot e revisão diretamente — a associação existe apenas por proximidade temporal e autoral. A quarta é que **alterações fora do fluxo formal não geram snapshot**: intervenções diretas no banco, ainda que possíveis para a equipe de suporte, não geram revisão nem snapshot, apenas atualizam `last_update`. A política de acesso ao banco em produção está fora do escopo do módulo, mas o efeito sobre a rastreabilidade precisa ser reconhecido.

Essas limitações não invalidam o mecanismo; delimitam o que ele garante. O sistema passou de um fluxo sem registro formal (Seção 3.1.4) para um fluxo em que toda operação realizada pela aplicação é registrada com autor, data, justificativa e estado anterior. O que está fora do fluxo formal — intervenções diretas no banco — permanece fora do escopo da garantia, e isso é uma decisão de escopo, não uma falha do mecanismo.

3.4 Testes e Validação

3.4.1 Construção da Infraestrutura de Testes

O sistema Banco de Especialistas não dispunha, antes do desenvolvimento deste módulo, de infraestrutura de testes automatizados. A verificação das funcionalidades era feita manualmente, por meio de requisições à API e inspeção do banco. Diante disso, a validação do módulo de solicitações exigiu a construção prévia de uma base de testes reutilizável, que pudesse ser aplicada não apenas ao módulo novo, mas a funcionalidades futuras.

A construção seguiu uma ordem deliberada. Primeiro, foi para funcionaldades básicas, que os novos módulos não deveria nem interferir, as features relacionadas a criação de especialistas, tratamento de documentos, pontuação e RBAC. Em seguida, foram implementados testes de regressão para as funcionalidades existentes que o novo módulo afeta ou consome: mudança de categoria (`test_change_category.py`), notificação (`test_notification.py`), usuário (`test_user.py`) e especialista (`test_specialist.py`). Esses testes estabeleceram uma linha de base — o comportamento que não poderia ser alterado. Só depois dessa linha de base, foram implementados os testes do próprio módulo de solicitações. A ordem não foi acidental: sem a linha de base, a refatoração introduzida pelo módulo novo não teria como ser verificada contra regressões.

No total, a suíte implementada contém 89 testes, distribuídos em 44 testes para o módulo novo e 45 testes de regressão. A proporção, próxima de 1:1, reflete a estratégia de equilíbrio entre adicionar cobertura para o que foi construído e preservar a cobertura do que existia. A suíte completa executa em aproximadamente três minutos, o que viabiliza sua integração à pipeline de verificação contínua.

A distribuição dos testes por arquivo é apresentada no Quadro 6.

Quadro 6 – Distribuição dos testes implementados.

Arquivo	**Testes**	**Propósito**
Módulo novo — fluxo completo		
`test_change_category.py`	19	Fluxo ponta a ponta: solicitação → revisão → mudança de categoria
Módulo novo — camadas		
`test_solicitation.py`	11	Integração pela API
`test_solicitation_service.py`	11	Camada de serviço
`test_review.py`	3	Integração pela API (decisão)
Regressão		
`test_notification.py`	22	Notificações — efeito colateral
`test_specialist.py`	8	Especialista — entidade modificada
`test_user.py`	7	Usuário — entidade associada
`test_stale_criteria_on_category_switch.py`	2	Critérios após mudança de categoria
`test_list_items_without_declaration.py`	2	Listagem de itens
`test_geral.py`	4	Verificações gerais
Total	**89**	

Fonte: Elaboração própria (2026).

Dois arquivos de regressão merecem destaque. O `test_stale_criteria_on_category_switch.py` e `test_list_items_without_declaration.py` cobrem um efeito colateral específico: a mudança de categoria pode tornar obsoletos os critérios de avaliação e a listagem de itens do especialista. Esses testes não verificam o módulo novo; verificam que o módulo novo não corrompe o que já existia ao alterar a categoria. São a evidência prática do RNF05.

O arquivo `test_change_category.py`, com 19 testes, é o mais relevante da suíte. Ele cobre o fluxo completo de mudança de categoria — criação da solicitação, aprovação pela rota de revisão, aplicação do efeito sobre o especialista e verificação do estado resultante —, para todas as transições válidas e para os casos de erro identificados na análise de regras de negócio (Seção 3.1.3). Diferentemente dos demais arquivos, que testam um módulo por vez, esses testes atravessam os três módulos envolvidos no fluxo — solicitação, revisão e especialista —, e são a evidência prática de que a integração entre eles funciona. É a este arquivo que o restante deste capítulo se refere quando afirma que a mudança de categoria só existe como integração (Seção **??**).

3.4.2 Isolamento entre Testes

O isolamento entre testes é obtido por `TRUNCATE` das tabelas transacionais antes e depois de cada teste. A escolha do `TRUNCATE` com `RESTART IDENTITY CASCADE`, em vez do padrão mais comum de *rollback* transacional, foi deliberada e decorre de uma característica específica do ambiente.

O padrão mais difundido para isolamento em testes com SQLAlchemy é abrir uma transação por teste e fazer *rollback* ao final. Esse padrão funciona quando todas as operações do teste compartilham a mesma sessão de banco. No entanto, o cliente de testes do Flask (`app.test_client()`) abre sessões próprias ao processar cada requisição, e essas sessões participam de transações independentes da transação do teste. Como consequência, o *rollback* da transação do teste não desfaz as alterações feitas pelas requisições HTTP — o isolamento falha.

O `TRUNCATE` resolve o problema porque opera no nível do banco, fora do controle transacional das sessões da aplicação. A limpeza acontece em uma conexão dedicada, com `commit` explícito, independente das sessões abertas pelo cliente de teste. O Algoritmo 6 descreve a função de limpeza em pseudocódigo.

Algoritmo 6: Limpeza de tabelas transacionais entre testes (pseudocódigo).

```
TABELAS_TRANSACIONAIS = [
    "specialists",
    "users",
    "solicitations",
    "solicitation_reviews",
    "specialist_snapshots",
    "notifications",
]

FUNÇÃO truncar(app, db):
    COM app.em_contexto():
        COM db.engine.conectar() COMO conexão:
            conexão.executar(
                "TRUNCATE TABLE "
                + juntar(TABELAS_TRANSACIONAIS)
                + " RESTART IDENTITY CASCADE")
            conexão.commit()

FIXTURE session(db, app):    // autouse, escopo de função
    truncar(app, db)
    ENTREGAR db.session
    truncar(app, db)
```

A escolha do `TRUNCATE` tem uma contrapartida: ele é mais lento que o *rollback*, porque envolve I/O real e reinício de sequências. Para a suíte do módulo de solicitações, com poucas

dezenas de testes, o custo é irrelevante. Para suítes maiores, o *rollback* transacional seria preferível, desde que o cliente de testes fosse adaptado para participar da transação — o que exigiria reescrever os testes em um estilo diferente. A escolha do `TRUNCATE` privilegia simplicidade e robustez sobre desempenho, o que é adequado ao estágio atual do projeto.

3.4.3 Fixtures de Autenticação

Os testes do módulo de solicitações exigem, em sua maioria, um usuário autenticado — solicitante ou avaliador. Sem abstração, cada teste precisaria criar o usuário, autenticar, obter o token e montar o cabeçalho de autorização, o que repetiria cerca de quinze linhas de código por teste. Para evitar essa repetição, foram implementadas duas fixtures de autenticação: `auth_specialist` e `auth_administrator`.

A fixture `auth_specialist` cria um usuário do tipo especialista com dados gerados por `UserFake`, autentica-o via `POST /users/login`, extrai o token da resposta e cria os dados pessoais do especialista via `POST /specialists/personal_data`. Retorna o cabeçalho de autorização pronto para ser usado pelo cliente de testes. A fixture aceita um payload opcional, o que permite que cada teste ajuste os dados do usuário sem duplicar a lógica de autenticação.

A fixture `auth_administrator` segue o mesmo padrão, mas cria um usuário com perfil de administração (`ProfileEnum.ADMIN`) e tipo de vínculo servidor. O identificador é fixo (`99999`), o que permite que a fixture seja idempotente: se o usuário já existe, ele é reaproveitado; se não, é criado. O perfil de administração é o que satisfaz o `FeatureEnum.SPECIALIST_REVIEW` exigido pelas rotas de revisão, conforme a Seção **??**.

A separação em duas fixtures, em vez de uma fixture parametrizada, justifica-se pela diferença de fluxo: o especialista precisa de dados pessoais para estar completo no sistema, enquanto o administrador precisa de perfil específico para acessar as rotas de revisão. Unificar em uma única fixture forçaria condicionais no corpo da própria fixture, o que prejudicaria a legibilidade. A duplicação de oito linhas entre as duas é preferível à complexidade de uma fixture condicional.

O uso das fixtures reduz o boilerplate nos testes e, mais importante, garante que a preparação de cada teste seja consistente: todos os testes que exigem autenticação passam pelo mesmo fluxo, o que elimina variações acidentais na preparação. Quando a lógica de autenticação mudar — por exemplo, se o formato do token mudar —, a correção é feita em um único lugar.

3.4.4 Testes de integração

Os testes de integração verificaram a comunicação entre os módulos e a correta interação entre backend, banco de dados e interfaces do sistema. Essa camada de validação foi importante para confirmar que as funcionalidades operavam de forma coerente no contexto completo da aplicação.

3.4.5 Processo de Integração Contínua

O processo de integração contínua foi adotado como mecanismo de validação recorrente, permitindo automatizar a execução dos testes e reduzir erros durante o desenvolvimento. Esse fluxo contribui para manter a qualidade do código e garantir que novas alterações não comprometam o funcionamento do sistema.

4 CONSIDERAÇÕES FINAIS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

Referências

BAYER, M. Sqlalchemy. In: BROWN, A.; WILSON, G. (Ed.). **The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks**. Lulu.com, 2012. Disponível em: <https://aosabook.org/en/v2/sqlalchemy.html>. Acesso em: 25 out. 2026. Citado na página 5.

DOCKER. **Docker Documentation**. [S.l.], 2024. Disponível em: <https://docs.docker.com/>. Acesso em: 26 out. 2026. Citado na página 7.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/info/rfc9110>. Acesso em: 25 out. 2026. Citado na página 4.

FLASGGER DEVELOPERS. **Flasgger: Easy OpenAPI Specs and Swagger UI for Your Flask API**. 2024. Disponível em: <https://github.com/flasgger/flasgger>. Acesso em: 27 out. 2026. Citado na página 5.

FOWLER, M. **Audit Log**. 2004. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/AuditLog.html>. Acesso em: 25 out. 2026. Citado na página 6.

FOWLER, M. **Event Sourcing**. 2005. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/EventSourcing.html>. Acesso em: 25 out. 2026. Citado na página 6.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, Massachusetts: Addison-Wesley, 1994. Citado na página 6.

Git. **Git: fast-version-control**. 2026. Disponível em: <https://git-scm.com/>. Acesso em: 27 out. 2026. Citado na página 7.

GitLab. **GitLab Documentation**. 2026. <https://docs.gitlab.com/>. Acesso em: 25 out. 2026. Citado na página 7.

HOLGER KREKEL AND PYTEST DEVELOPERS. **pytest: helps you write better programs**. [S.l.], 2024. Disponível em: <https://docs.pytest.org/>. Acesso em: 25 out. 2026. Citado na página 7.

KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 6.

MARSHMALLOW DEVELOPERS. **marshmallow: simplified object serialization**. [S.l.], 2024. Disponível em: <https://marshmallow.readthedocs.io/>. Acesso em: 25 out. 2026. Citado na página 5.

OpenProject. **OpenProject Documentation**. 2026. <https://www.openproject.org/docs/>. Acesso em: 25 out. 2026. Citado na página 3.

PALLETS. **Flask Documentation**. [S.l.], 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 25 out. 2026. Citado na página 4.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. [S.l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 out. 2026. Citado na página 5.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado 2 vezes nas páginas 6 e 7.

TANENBAUM, A. S.; van Steen, M. **Sistemas Distribuídos: Princípios e Paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007. Citado na página 4.

Apêndices

APÊNDICE A – Nome do apêndice

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros apêndices para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do apêndice.

Não é aconselhável colocar tudo que é complementar em um único apêndice. Organize os apêndices de modo que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho.

APÊNDICE B – Nome do outro apêndice

conteúdo do novo apêndice

Anexos

ANEXO A – Nome do anexo

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros anexos para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do anexo.

Organize seus anexos de modo a que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho. É para ele que você escreve.

ANEXO B – Nome do outro anexo

conteúdo do outro anexo